

Uni-FACEF CENTRO UNIVERSITÁRIO MUNICIPAL DE FRANCA

**PERSONAL GUIDE: SOFTWARE PARA GESTÃO DE CLIENTE E
TREINOS PARA PERSONAL TRAINER**

**GUILHERME DE OLIVEIRA PARREIRA
LUCAS ANTONIO MONTEIRO
MATHEUS LOMBARDI DE SOUZA
RAFAEL MENDES FERREIRA**

**FRANCA
2026**

Sumário

1. Introdução	3
2. Levantamento de requisitos.....	4
3. Modelo Conceitual	6
4. Modelagem Lógica	8
5. Modelagem Física	10
6. Álgebra Relacional	13

1. Introdução

Com o avanço da tecnologia e a crescente digitalização de processos, torna-se cada vez mais importante o desenvolvimento de sistemas que auxiliem profissionais na organização e otimização de suas atividades. Nesse contexto, o presente trabalho propõe o desenvolvimento de um sistema voltado para atender às necessidades de um personal trainer, com foco na gestão de alunos e na elaboração de treinos.

Atualmente, o profissional realiza essas atividades por meio de planilhas e anotações manuais, o que torna o processo mais demorado e sujeito a erros. A criação dos treinos exige tempo considerável, além de dificultar a padronização e o acompanhamento da evolução dos alunos. Da mesma forma, a ausência de uma ferramenta centralizada compromete a organização das informações, como dados pessoais, planos e histórico de treinos.

Diante desse cenário, identifica-se como principal problema a ineficiência no gerenciamento dos alunos e na elaboração dos treinos, impactando diretamente na produtividade e na qualidade do serviço prestado.

Assim, o objetivo deste sistema é automatizar e organizar esses processos, permitindo a criação mais rápida de treinos e o armazenamento centralizado das informações dos alunos. Com isso, busca-se proporcionar maior controle, agilidade e eficiência no dia a dia do profissional, contribuindo para um melhor acompanhamento dos clientes.

2. Levantamento de requisitos

Nesta seção são apresentados os requisitos do sistema, que representam as funcionalidades e restrições necessárias para atender às demandas do personal trainer. O levantamento foi realizado com base na análise do problema identificado, buscando estruturar as principais operações que o sistema deve oferecer. Os requisitos funcionais descrevem as ações que o sistema precisa executar, garantindo suporte adequado à gestão de alunos, treinos e acompanhamento de desempenho.

REQUISITOS FUNCIONAIS (RF)	
ID: RF001	Nome do Requisito: Cadastrar Alunos
Descrição ->	O sistema deve permitir o registro de novos alunos, centralizando os dados que hoje estão fragmentados entre Google Sheets e apps de terceiros.
Categoria: Evidente	Prioridades: Essencial
Informações ->	ID, Nome, Data de Nascimento, Endereço, Tipo, Telefone, Ativo, Status Pagamento
Regra de Negócio ->	O sistema não deve permitir CPFs duplicados.
ID: RF002	Nome do Requisito: Cadastrar Treinos
Descrição ->	O sistema deve automatizar a criação de planilhas semanais de forma interativa para reduzir o tempo operacional do profissional
Categoria: Evidente	Prioridades: Essencial
Informações ->	ID Treino, Nome do Aluno, Data, Lista de Exercícios, Séries, Repetições, Carga, Intensidade, Descanso
Regra de Negócio ->	Não há.
ID: RF003	Nome do Requisito: Cadastrar Exercícios
Descrição ->	O sistema deve manter um cadastro de exercícios com orientações técnicas para evitar a redigitação manual frequente
Categoria: Evidente	Prioridades: Essencial
Informações ->	ID Exercício, Nome, Descrição
Regra de Negócio ->	Não é permitido o cadastro do exercício com o mesmo nome.

ID: RF004	Nome do Requisito: Agendar Aulas Presenciais
Descrição ->	O sistema deve permitir o controle de horários e presenças dos alunos presenciais na agenda do profissional.
Categoria: Evidente	Prioridades: Importante
Informações ->	ID, Aluno, Data/Hora, Check-in
Regra de Negócio ->	A exclusão de um aluno presencial implica na anulação automática de todos os seus horários agendados.

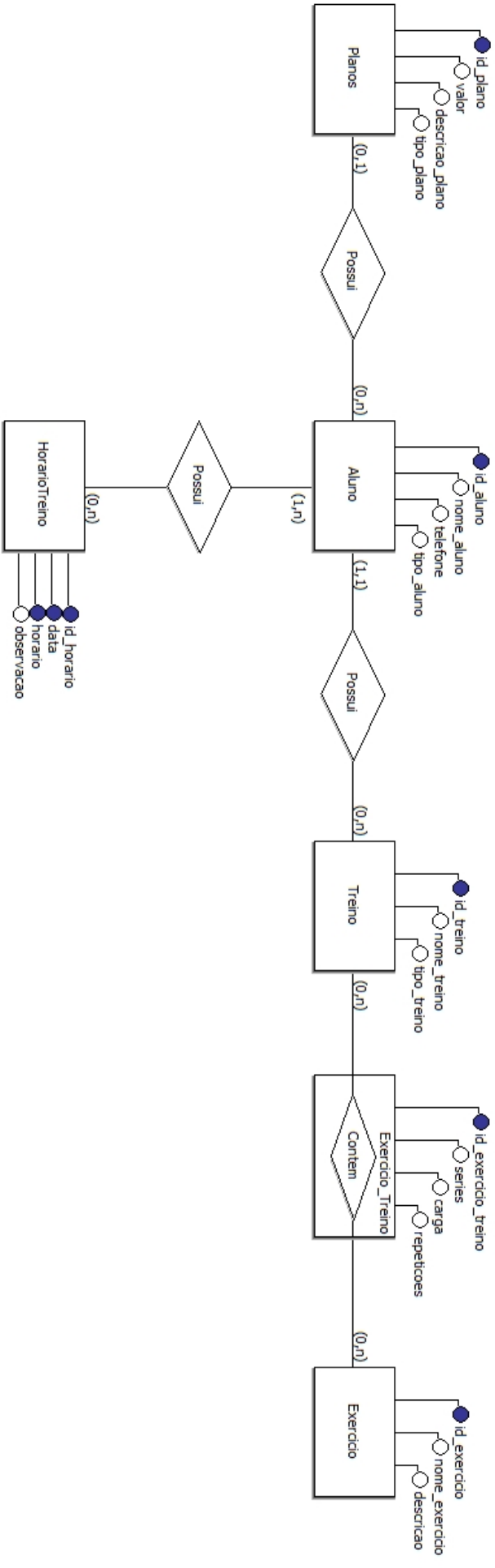
ID: RF005	Nome do Requisito: Vincular Treino ao Aluno
Descrição ->	O sistema deve permitir que um treino montado seja associado a um aluno específico da base cadastrada.
Categoria: Evidente	Prioridades: Essencial
Informações ->	ID do Treino, ID do Aluno.
Regra de Negócio ->	Um treino só pode ser vinculado após a consulta e confirmação da existência do aluno na base de dados.

ID: RF006	Nome do Requisito: Registrar Desempenho e Carga
Descrição ->	O sistema deve permitir que o profissional anote o progresso de carga e repetições para manter o histórico de evolução do aluno.
Categoria: Evidente	Prioridades: Importante
Informações ->	Peso/Carga, Número de Repetições, Potência
Regra de Negócio ->	Não há.

ID: RF007	Nome do Requisito: Registrar Presença
Descrição ->	O sistema deve permitir o check-in das aulas para controle de frequência e assiduidade dos alunos presenciais.
Categoria: Evidente	Prioridades: Importante
Informações ->	ID Aluno, Data/Hora, Status (Check-in).
Regra de Negócio ->	Permite registrar faltas justificadas ou agendar/remarcar aulas conforme disponibilidade na agenda.

3. Modelo Conceitual

O modelo conceitual tem como objetivo representar, de forma abstrata, a estrutura dos dados do sistema e o relacionamento entre as entidades envolvidas. Nesta etapa, são identificados os principais elementos do domínio, como alunos, treinos e exercícios, bem como suas interações. Essa modelagem permite uma visão clara do funcionamento do sistema, independente de tecnologias específicas de implementação.

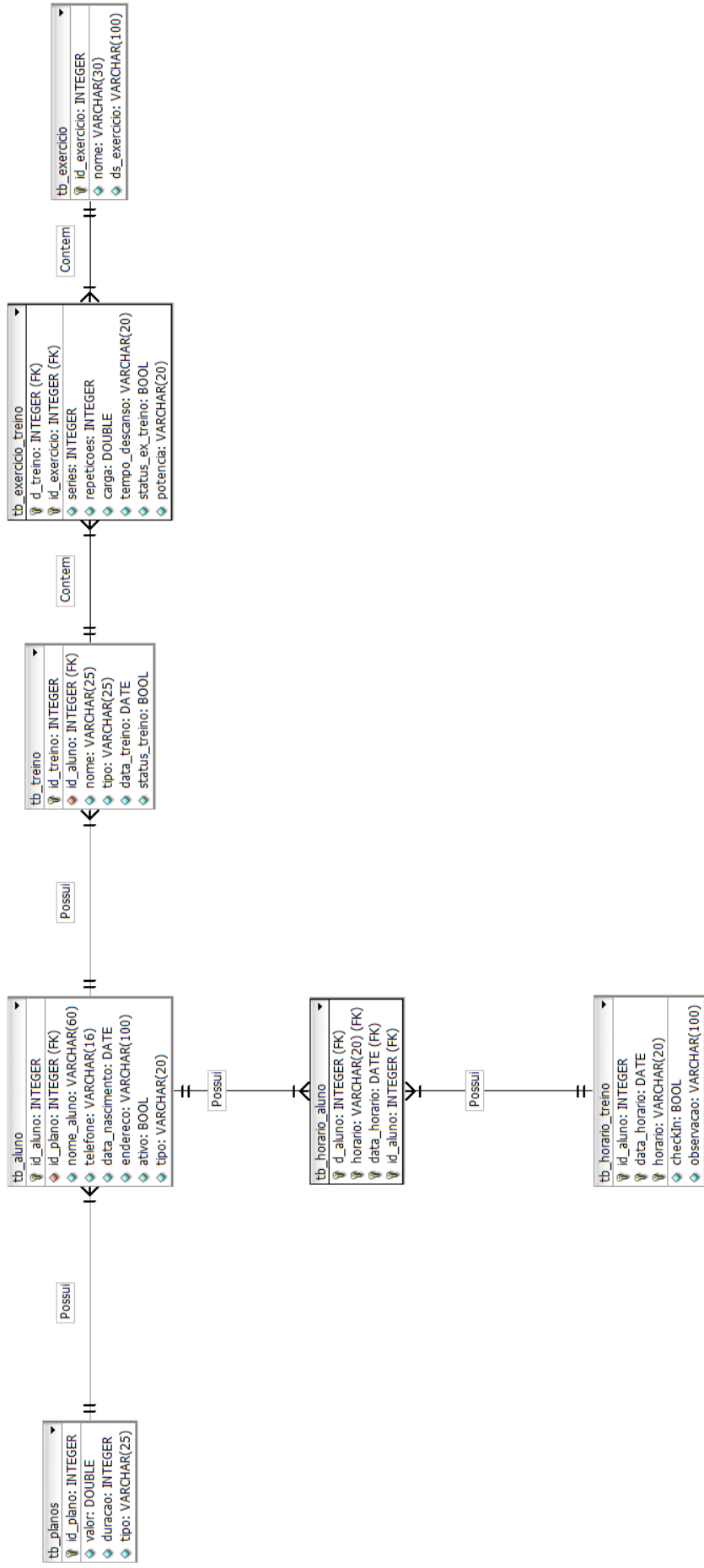


4. Modelagem Lógica

A modelagem lógica traduz o modelo conceitual para uma estrutura mais próxima da implementação em banco de dados relacional. Nessa fase, as entidades e relacionamentos são convertidos em tabelas, atributos e chaves, respeitando regras de normalização e integridade. O objetivo é garantir consistência, organização e eficiência no armazenamento dos dados.

Modelagem:

- tb_planos (id_plano (pk), valor, descricao, duracao, tipo)
- tb_aluno(id_aluno (pk), nome, telefone, data nascimento, endereco, status, tipo, id_plano (fk))
- tb_horario_treino(id_horario (pk), data (pk), horário (pk), checkIn, observacao, id_aluno(fk))
- tb_exercicio(id_exercicio (pk), nome, descricao)
- tb_treino(id_treino (pk), nome, tipo, data, status, id_aluno (fk) (NN))
- tb_exercicio_treino(id_exercicio_treino (pk), potencia, intensidade, series, repetições, carga, tempo descanso, status, id_treino (fk), id_exercicio (fk))



5. Modelagem Física

A modelagem física corresponde à implementação do banco de dados em um Sistema Gerenciador de Banco de Dados (SGBD), sendo o escolhido o PostgreSQL. Nesta seção, são apresentadas as instruções SQL responsáveis pela criação das tabelas, definição de chaves primárias e estrangeiras, além de restrições de integridade. Essa etapa materializa toda a estrutura planejada nas fases anteriores.

```
CREATE TABLE tb_planos(  
    id_plano      INTEGER,  
    valor         DOUBLE PRECISION,  
    duracao       INTEGER,  
    tipo          VARCHAR(25),  
  
    CONSTRAINT pk_tb_planos_id_plano  
        PRIMARY KEY(id_plano)  
);
```

```
CREATE TABLE tb_aluno(  
    id_aluno      INTEGER,  
    id_plano      INTEGER,  
    nome          VARCHAR(60) CONSTRAINT nn_tb_aluno_nome NOT NULL,  
    telefone      VARCHAR(16),  
    data_nascimento DATE,  
    endereco      VARCHAR(100),  
    ativo         BOOLEAN,  
    tipo          VARCHAR(20),  
  
    CONSTRAINT pk_tb_aluno_id_aluno  
        PRIMARY KEY(id_aluno),  
  
    CONSTRAINT fk_tb_aluno_id_plano  
        FOREIGN KEY(id_plano)  
        REFERENCES tb_planos(id_plano)  
);
```

```
CREATE TABLE tb_treino(  
    id_treino     INTEGER,  
    id_aluno      INTEGER,  
    nome          VARCHAR(25),  
    tipo          VARCHAR(25),  
    data_treino   DATE,  
    status_treino BOOLEAN,
```

```

CONSTRAINT pk_tb_treino_id_treino
    PRIMARY KEY(id_treino),

CONSTRAINT fk_tb_treino_id_aluno
    FOREIGN KEY(id_aluno)
    REFERENCES tb_aluno(id_aluno)
);
CREATE TABLE tb_exercicio(
    id_exercicio    INTEGER,
    nome            VARCHAR(30),
    ds_exercicio    VARCHAR(100),

    CONSTRAINT pk_tb_exercicio_id_exercicio
        PRIMARY KEY(id_exercicio)
);

```

```

CREATE TABLE tb_exercicio_treino(
    id_treino        INTEGER,
    id_exercicio     INTEGER,
    series           INTEGER,
    repeticoes       INTEGER,
    carga            DOUBLE PRECISION,
    tempo_descanso   VARCHAR(20),
    status_ex_treino BOOLEAN,
    potencia         VARCHAR(20),

    CONSTRAINT pk_tb_exercicio_treino_ids
        PRIMARY KEY(id_treino, id_exercicio),

    CONSTRAINT fk_tb_ex_treino_id_treino
        FOREIGN KEY(id_treino)
        REFERENCES tb_treino(id_treino),

    CONSTRAINT fk_tb_ex_treino_id_exercicio
        FOREIGN KEY(id_exercicio)
        REFERENCES tb_exercicio(id_exercicio)
);

```

```

CREATE TABLE tb_horario_aluno(
    id_aluno        INTEGER,
    horario         VARCHAR(20),
    data_horario    DATE,

    CONSTRAINT pk_tb_horario_aluno
        PRIMARY KEY(id_aluno, horario, data_horario),

```

```
CONSTRAINT fk_tb_horario_aluno_id_aluno  
  FOREIGN KEY(id_aluno)  
  REFERENCES tb_aluno(id_aluno)  
);
```

```
CREATE TABLE tb_horario_treino(  
  id_aluno      INTEGER,  
  data_horario  DATE,  
  horario       VARCHAR(20),  
  checkin      BOOLEAN,  
  observacao    VARCHAR(100),  
  
  CONSTRAINT pk_tb_horario_treino  
    PRIMARY KEY(id_aluno, data_horario, horario),  
  
  CONSTRAINT fk_tb_horario_treino_id_aluno  
    FOREIGN KEY(id_aluno)  
    REFERENCES tb_aluno(id_aluno)  
);
```

6. Álgebra Relacional

A álgebra relacional é utilizada para representar formalmente as operações realizadas sobre os dados armazenados no banco. Nesta seção, são demonstradas consultas que exemplificam a extração de informações relevantes do sistema, como relatórios de alunos, treinos e exercícios. Essas operações evidenciam como os dados podem ser manipulados e analisados para apoiar a tomada de decisão do profissional.

1. Relatório: Alunos ativos cadastrados no sistema

```
R1 <-  $\sigma$  (ativo = TRUE) (tb_aluno)
```

```
R2 <-  $\pi$  (id_aluno, nome, telefone) (R1)
```

2. Relatório: Alunos com seus respectivos planos

```
R1 <- tb_aluno  $\bowtie$  tb_planos
```

```
R2 <-  $\pi$  (nome, tipo, valor) (R1)
```

3. Relatório: Treinos cadastrados por aluno

```
R1 <- tb_aluno  $\bowtie$  tb_treino
```

```
R2 <-  $\pi$  (tb_aluno.nome, tb_treino.nome, data_treino) (R1)
```

4. Relatório: Exercícios de cada treino

```
R1 <- tb_exercicio_treino  $\bowtie$  tb_exercicio
```

```
R2 <-  $\pi$  (id_treino, tb_exercicio.nome, series, repeticoes) (R1)
```

5. Relatório: Alunos que possuem treinos ativos

```
R1 <- tb_aluno  $\bowtie$  tb_treino
```

```
R2 <-  $\sigma$  (status_treino = TRUE) (R1)
```

```
R3 <-  $\pi$  (tb_aluno.nome, tb_treino.nome) (R2)
```